

Roadmap de Implementação

Trabalho Final – Agente de EDA
Guia dia a dia para os 15 dias do projeto

Duração total: 15 dias corridos
Esforço estimado: 25–35h por aluno
Grupo: 2 a 4 alunos

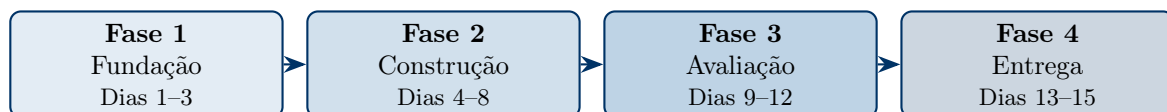
Como usar este roadmap

Este documento divide os 15 dias em **quatro fases** com objetivos claros e **checkpoints diários**. Cada checkpoint é uma pequena entrega que prova que vocês estão prontos para o dia seguinte.

Importante: os tempos sugeridos por dia (1–3 horas) assumem que vocês estão trabalhando *em paralelo* com outras disciplinas. Se atrasar 1 dia, é recuperável. Se atrasar 3 dias, vocês entregam incompleto.

A regra dos 3 dias. Se travarem por mais de 3 dias na mesma etapa, **procurem o professor**. Não fiquem “só mais um dia tentando” — o tempo é curto demais para isso.

Visão geral das fases



Fase	Objetivo
1. Fundação	Setup do ambiente, escolha do dataset, primeira chamada à API funcionando
2. Construção	Agente completo, com todas as tools obrigatórias rodando
3. Avaliação	Benchmark de 30+ perguntas executado e analisado
4. Entrega	Relatório escrito, slides prontos, ensaio da apresentação

1 Fase 1 – Fundação (Dias 1–3)

Objetivo da fase

Sair desta fase com o projeto rodando no PyCharm, dataset escolhido e validado, e a **primeira chamada à API do LLM funcionando** (mesmo que sem tools).

DIA 1 Setup do ambiente e divisão de papéis

Tempo: 2–3 horas. Pode ser dividido entre os integrantes.

Tarefas:

1. Descompactem o `projeto_agente_edu.zip` fornecido pelo professor.
2. Abram a pasta no PyCharm: *File* → *Open*.
3. Aceitem a sugestão do PyCharm de criar um `venv`.
4. Instalem as dependências:

```
pip install -r requirements.txt
```

5. Marquem a pasta raiz como *Sources Root* (clique direito na pasta).
6. Cada integrante cria conta na Anthropic e gera uma API key em <https://console.anthropic.com/> (ou no provedor escolhido).
7. Criem o repositório Git no GitHub/GitLab e adicionem todos como colaboradores.

Definam papéis no grupo (podem se sobrepor, mas alguém “puxa” cada área):

- **Tech lead** – responsável pelo agente e pela integração com a API
- **Data/QA** – responsável pelo dataset, benchmark e métricas
- **Documentação** – responsável pelo relatório e pelos slides
- (Se grupo de 4) **Tools/Visualização** – responsável pelas tools extras e gráficos

Checkpoint

- ☐ Todos os integrantes conseguem abrir o projeto no PyCharm
- ☐ `python -m pytest tests/` roda e os 14 testes passam
- ☐ Repositório Git criado e compartilhado com o professor
- ☐ Cada um tem sua própria API key (não compartilhem! cada um pode ter créditos grátis)

Armadilha comum

Não pulem o `venv`. Instalar pacotes globalmente vai gerar conflitos com outras disciplinas mais tarde. Se o PyCharm não criar automaticamente, vejam o tutorial no `README.md`.

DIA 2 Escolha e exploração do dataset

Tempo: 2–3 horas.

Tarefas:

1. Discutam em grupo qual dataset usar entre as opções do enunciado.

2. Baixem o CSV escolhido e coloquem em `data/`.
3. Editem `config.py`, ajustando `DATASET_PATH` para apontar para o arquivo de vocês.
4. Façam uma análise exploratória manual em um Jupyter Notebook ou script:
 - Quantas linhas? Quantas colunas?
 - Quais são as colunas numéricas? Quais são as categóricas?
 - Tem valores nulos? Quantos?
 - Quais as 5 perguntas mais interessantes que dá para fazer sobre esses dados?
5. Documentem essas respostas — vão usar no relatório depois.

Critério para escolher o dataset: prefiram um que vocês ENTENDAM o domínio. Se ninguém do grupo entende o que cada coluna significa, vão ter dificuldade em criar perguntas de avaliação que façam sentido.

Checkpoint

- ☐ Dataset em `data/` com pelo menos 6 colunas e 1.000 linhas
- ☐ `config.py` ajustado
- ☐ Documento curto (1 página) descrevendo o dataset
- ☐ 5 perguntas-rascunho que vocês querem que o agente saiba responder

DIA 3 Primeira chamada à API funcionando

Tempo: 2–3 horas.

Tarefas:

1. Copiem `.env.example` para `.env` e preencham com sua API key.
2. Rodem a CLI:

```
python cli.py
```

3. Façam 3 perguntas simples ao agente e observem:
 - “Quantas linhas tem o dataset?”
 - “Quais são as colunas?”
 - “Qual a média de [uma coluna numérica do dataset]?”
4. Usem o comando `/trajetoria` dentro da CLI para ver quais tools o agente chamou.
5. Usem o comando `/custo` para ver quantos tokens foram gastos.

Checkpoint

- ☐ CLI roda sem erros e responde às 3 perguntas básicas
- ☐ Trajetória mostra que tools foram realmente chamadas
- ☐ Todos os integrantes conseguem rodar a CLI nas suas próprias máquinas

Armadilha comum

Se aparecer “ANTHROPIC_API_KEY não definida”, conferiram se:

- (a) o arquivo se chama `.env` (com ponto na frente) e está na raiz?
- (b) não tem espaços ao redor do `=`?
- (c) a chave começa com `sk-ant-`?

2 Fase 2 – Construção (Dias 4–8)

Objetivo da fase

Sair desta fase com o agente **completo e robusto**: todas as tools obrigatórias funcionando, pelo menos uma tool extra implementada, e o agente lidando bem com perguntas em múltiplos passos.

DIA 4 Completar a tool faltante (z-score)

Tempo: 1–2 horas.

Tarefas:

1. Abram `tools/stats_tools.py` e localizem o TODO no `detectar_outliers` para o método `zscore`.
2. Implementem:
 - Calcular média e desvio-padrão da série
 - Calcular o z-score de cada valor: $z = (x - \bar{x})/\sigma$
 - Considerar outlier se $|z| > 3$
 - Retornar um dict no MESMO FORMATO que o método IQR
3. Escrevam um teste em `tests/test_tools.py`:

```
def test_detectar_outliers_zscore_funciona():
    resultado = detectar_outliers("idade", metodo="zscore")
    assert "erro" not in resultado
    assert "total_outliers" in resultado
```

4. Rodem `pytest tests/` e confirmem que passa.

Por que esta tarefa primeiro? É a forma mais simples de vocês entenderem o código antes de mexer no agente. Se conseguirem fazer isso, conseguem fazer o resto.

Checkpoint

- ☐ Método z-score implementado
- ☐ Teste novo passando
- ☐ Agente consegue responder “há outliers em [coluna] usando z-score?”

DIA 5 Criar a primeira tool extra

Tempo: 2–3 horas.

Tarefas:

1. Conversem em grupo: *qual operação está faltando que faria sentido para o nosso dataset?* Algumas ideias:
 - `calcular_percentil(coluna, percentil)` — útil para análise de salários, notas, etc.
 - `filtrar_por_data(coluna_data, inicio, fim)` — se o dataset tem séries temporais
 - `tabela_contingencia(col_a, col_b)` — cruzar duas categóricas
 - `contar_nulos()` — visão geral de qualidade dos dados

- `teste_hipotese(grupo_a, grupo_b, coluna)` — comparar duas amostras
2. Criem um novo arquivo `tools/extra_tools.py` (ou adicionem em um arquivo existente).
 3. Implementem a tool com o decorador `@tool` (copiem o padrão das outras).
 4. Adicionem o import em `tools/__init__.py`.
 5. Escrevam pelo menos 2 testes para a nova tool.
 6. Testem na CLI: o agente percebe que pode usar a nova tool quando faz sentido?

Esta é uma das tarefas que mais pontua na rubrica. Pensem bem: uma tool extra bem justificada vale mais que três tools extras genéricas. Documentem POR QUÊ vocês escolheram essa.

Checkpoint

- ☐ Pelo menos 1 tool extra implementada e testada
- ☐ Agente usa a nova tool quando aplicável (testem com pergunta direcionada)
- ☐ Justificativa de 1 parágrafo escrita no relatório-rascunho

DIA 6 Stress test do agente – perguntas difíceis

Tempo: 2–3 horas.

Tarefas:

1. Cada integrante prepara 5 perguntas difíceis para fazer ao agente:
 - Perguntas com múltiplos filtros (“mulheres acima de 40 anos com renda...”)
 - Perguntas que exigem 2–3 tools encadeadas
 - Perguntas ambíguas (“qual é melhor?”)
 - Perguntas sobre colunas que NÃO existem (testar comportamento de erro)
 - Perguntas em português coloquial (“e aí, tem mais homem ou mulher?”)
2. Rodem todas no agente e ANOTEM:
 - Quais funcionaram bem?
 - Quais falharam? Por quê?
 - O agente alucinou alguma coluna ou valor?
3. Salvem 3 trajetórias completas (com `/trajetoria`) em um arquivo de notas — vão usar no relatório.

Esta etapa é a mais valiosa de toda a Fase 2. Aqui vocês descobrem os PROBLEMAS REAIS do agente. Tudo que vocês listarem aqui vira material para a “análise de trajetórias e erros” do relatório.

Checkpoint

- ☐ Pelo menos 20 perguntas testadas
- ☐ Lista escrita de problemas observados
- ☐ 3 trajetórias salvas para análise futura

DIA 7 Refinamento: prompt e tratamento de erros

Tempo: 2–3 horas.

Tarefas:

1. Com base nos problemas do dia anterior, refinem o `SYSTEM_PROMPT` em `agent/agent.py`.
 - O agente está chamando tools demais? Adicionem uma diretriz para economizar.
 - Está alucinando colunas? Reforcem a diretriz de usar nomes exatos.
 - Está respondendo coisas demais? Adicionem diretriz de concisão.
2. Melhorem o tratamento de erros nas tools onde for necessário.
3. Re-rodem as 20 perguntas do dia anterior. Quantas melhoraram?

Armadilha comum

Não façam o prompt enorme. Quanto mais texto no system prompt, mais caro fica cada chamada. Sejam cirúrgicos: corrijam UM problema por vez e meçam o impacto. Mantenham o prompt original em um comentário para comparar.

Checkpoint

- ☐ System prompt refinado com pelo menos 2 melhorias justificadas
- ☐ Tools com tratamento de erro robusto (testes de erro passando)
- ☐ Documento de notas listando as melhorias feitas e o efeito de cada uma

DIA 8 Polimento final do agente + commit estável

Tempo: 1–2 horas.

Tarefas:

1. Limpem o código: removam `print()` de debug, comentários antigos, código morto.
2. Confiram que todos os testes ainda passam.
3. Façam um commit marcado: `git tag v1.0-agente-pronto`.
4. Este é o estado em que vocês começarão a avaliação na próxima fase.

Checkpoint

- ☐ Tag `v1.0-agente-pronto` no Git
- ☐ Todos os testes passando
- ☐ README atualizado com instruções específicas do grupo

3 Fase 3 – Avaliação (Dias 9–12)

Objetivo da fase

Sair desta fase com o **benchmark completo executado** e métricas quantitativas e qualitativas analisadas. Esta é a fase que mais pesa na rubrica (30% combinados entre benchmark e análise).

DIA 9 Construir o benchmark.json (parte 1: factuais)

Tempo: 2–3 horas. **Trabalho em paralelo:** dividam as perguntas entre os integrantes.

Tarefas:

1. Abram `evaluation/benchmark.json`.
2. Removam as 5 perguntas de exemplo.
3. Criem as **10 perguntas factuais simples**, uma única tool basta:
 - Quantas linhas tem o dataset?
 - Quais são as colunas?
 - Qual o valor mínimo de [coluna]?
 - Qual o valor máximo de [coluna]?
 - Quantos valores únicos tem [coluna categórica]?
 - etc.
4. Para CADA pergunta, calculem o gabarito **em pandas, no Jupyter**, e copiem o valor exato para o JSON.

```
import pandas as pd
df = pd.read_csv("data/seu_dataset.csv")
df.shape[0]          # resposta para "quantas linhas"
df["coluna"].min()    # resposta para "qual o min"
df["coluna"].nunique() # resposta para "valores unicos"
```

Armadilha comum

Erro mais comum aqui: colocar o gabarito “de cabeça” em vez de calcular em pandas. Depois descobrem que o agente está certo e o gabarito é que estava errado. **Sempre calculem em pandas** e colem o valor.

Checkpoint

- ☐ 10 perguntas factuais com gabarito calculado em pandas
- ☐ Cada pergunta tem id, tipo, pergunta, resposta_esperada, tipo_resposta

DIA 10 Construir o benchmark.json (parte 2: analíticas e ambíguas)

Tempo: 2–3 horas.

Tarefas:

1. Criem as **15 perguntas analíticas** (exigem 2+ tools encadeadas):

- Qual a [estatística] de [coluna] agrupada por [categoria]?
- Existe correlação entre [num1] e [num2]?
- Quantos [filtro] satisfazem [condição]?
- Há outliers em [coluna] quando consideramos apenas [filtro]?

2. Criem as **5 perguntas ambíguas/inválidas**:

- Qual a melhor coluna? (subjetivo)
- Por que [coluna] varia tanto? (causal, não é factual)
- Qual a renda média em 2050? (extrapolação inválida)
- Calcule a média de [coluna que não existe]

início do dataset

3. Para as ambíguas, `resposta_esperada:` "recusa" e `tipo_resposta:` "categorica".

Checkpoint

- ☐ Total de **30+** perguntas no benchmark
- ☐ Distribuição: 10 factuais, 15 analíticas, 5 ambíguas
- ☐ JSON válido (testem com `python -c "import json; json.load(open('evaluation/benchmark.json'))"`)

DIA 11 Rodar o benchmark e analisar métricas

Tempo: 2–3 horas (incluindo o tempo do agente rodando).

Tarefas:

1. Rodem o benchmark completo:

```
python -m evaluation.benchmark
```

2. Esperem terminar. Vai gerar um JSON em `logs/benchmark_<timestamp>.json`.

3. Anotem as **métricas quantitativas** obtidas:

- Acurácia geral
- Acurácia por tipo de pergunta
- Tool calls médio
- Latência média
- Tokens totais (e custo em US\$ – calculem manualmente)

4. Façam uma análise rápida em grupo:

- Que tipo de pergunta o agente acerta mais?
- Onde ele falha sistematicamente?
- As ambíguas funcionaram? Ou o agente tentou responder mesmo assim?

Cálculo de custo (Claude Haiku, valores aproximados de 2026):

Input: \$ 0,80 por milhão de tokens

Output: \$ 4,00 por milhão de tokens

Pesquisem o preço atualizado em <https://www.anthropic.com/pricing> antes de finalizar.**Checkpoint**

- ☐ Benchmark completo executado, log salvo
- ☐ Planilha (ou tabela em texto) com todas as métricas
- ☐ Lista preliminar de padrões de erro identificados

DIA 12 Análise de trajetórias e erros (parte qualitativa)**Tempo:** 2–3 horas.**Tarefas:**

1. Do log do benchmark, escolham **3 perguntas para análise detalhada**:
 - 1 trajetória de sucesso “bonita” (multi-tool, raciocínio bem encadeado)
 - 1 trajetória de erro interessante (agente errou de forma reveladora)
 - 1 caso ambíguo (como o agente lidou com a ambiguidade?)
2. Para cada uma, escrevam:
 - A pergunta original
 - Quais tools foram chamadas, em qual ordem
 - O que o agente respondeu
 - Análise: por que funcionou (ou falhou)?
3. Identifiquem **pelo menos 3 erros sistemáticos**:
 - Tipo de erro
 - Quantas vezes aconteceu
 - Hipótese da causa
 - Possível mitigação

Checkpoint

- ☐ 3 trajetórias detalhadamente comentadas (em arquivo de notas)
- ☐ 3+ erros sistemáticos documentados com hipóteses
- ☐ Discussão sobre alucinação registrada

4 Fase 4 – Entrega (Dias 13–15)

Objetivo da fase

Transformar tudo que vocês fizeram em uma **entrega coesa**: relatório escrito, repositório limpo, slides prontos e a apresentação ensaiada.

DIA 13 Escrever o relatório

Tempo: 3–4 horas. **Pode ser dividido:** cada integrante escreve uma seção.

Tarefas:

1. Sigam a estrutura solicitada no enunciado (até 12 páginas):

Seção	De onde vem o conteúdo	Páginas
1. Introdução	Dia 2 + dia 3	1
2. Dataset	Dia 2	1
3. Arquitetura do agente	Diagrama + decisões dos dias 4–7	2
4. Tools implementadas	Dias 4–5	1
5. Metodologia de avaliação	Dias 9–10 (descrição do benchmark)	1
6. Resultados quantitativos	Dia 11	1–2
7. Análise qualitativa	Dia 12 (trajetórias e erros)	2
8. Discussão crítica	Custo, latência, alucinação, viés, ética, limitações	1–2
9. Conclusão e trabalhos futuros	Síntese	1

A análise crítica (seções 7 e 8) vale 25% da nota. Não deixem para o fim nem façam superficial. É aqui que vocês mostram maturidade técnica.

Checkpoint

- ☐ Rascunho completo do relatório (todas as seções escritas)
- ☐ Diagrama da arquitetura desenhado (pode usar draw.io, Excalidraw, ou TikZ)
- ☐ Tabelas e gráficos das métricas inseridos

DIA 14 Slides + limpeza do repositório

Tempo: 3–4 horas.

Tarefas (em paralelo):

Pessoa A – Slides:

1. Abram o `template_apresentacao_alunos.pptx` fornecido pelo professor.
2. Preencham todos os placeholders (procurem por colchetes []).
3. Inseriram os gráficos reais nos slides 8 e 9.
4. Removam as “dicas em laranja” antes de apresentar (são só para vocês).

Pessoa B – Repositório:

1. Limpem arquivos de debug, `print()`, comentários antigos.
2. Confirmam que `.env` está no `.gitignore` (NUNCA committem a API key!).

3. Atualizem o `README.md` com:

- Nome do grupo e integrantes
- Dataset escolhido
- Como rodar o projeto
- Resultados principais (linkando para o relatório)

4. Façam o commit final e a tag `v1.0-final`.

Pessoa C – Revisão do relatório:

1. Releiam todas as seções procurando inconsistências.
2. Confiram que todos os números no relatório batem com o log do benchmark.
3. Confiram a contagem de páginas (até 12).
4. Exportem para PDF.

Checkpoint

- ☐ Slides preenchidos, sem placeholders nem dicas em laranja
- ☐ Repositório limpo, README atualizado, tag final criada
- ☐ Relatório em PDF revisado e dentro do limite de páginas
- ☐ `.env` NÃO está commitado (confiram com `git log - .env`)

Armadilha comum

Verifiquem se a API key não vazou. Se vocês acidentalmente commitaram o `.env`, REVOGUEM a chave no console da Anthropic IMEDIATAMENTE e gerem uma nova. Isso já causou cobranças não autorizadas em projetos passados.

DIA 15 Ensaio da apresentação e ajustes finais

Tempo: 2–3 horas.

Tarefas:

1. Façam pelo menos **2 ensaios completos** cronometrados.
2. Meta: **15 minutos** (a banca corta se ultrapassar muito).
3. Decidam quem fala o quê:

Slides	Conteúdo	Tempo
1–3	Capa, agenda, problema	2 min
4–6	Dataset, arquitetura, tools	3 min
7	Demo ao vivo	4 min
8–9	Benchmark e resultados	3 min
10–12	Análise qualitativa, crítica, conclusões	3 min

4. **Preparem a demo** com cuidado:

- Tenham 4 perguntas prontas (factual / analítica / com gráfico / ambígua)
- Treinem com a conexão da sala se possível

- Tenham um plano B: print de uma execução prévia salvo, caso a internet falhe
5. Preparem-se para perguntas da banca:
- “Por que escolheram esse modelo de LLM?”
 - “Por que essa tool extra e não outra?”
 - “Como vocês saberiam que a métrica de avaliação está boa?”
 - “O que mudariam se tivessem mais 15 dias?”
 - “Qualquer linha do código pode ser questionada”

Não decoram. Se decorarem e esquecerem uma palavra, travam. Saibam o conteúdo e falem naturalmente.

Checkpoint

- ☐ Pelo menos 2 ensaios completos feitos
- ☐ Tempo total dentro de 15 minutos
- ☐ Demo ensaiada com plano B preparado
- ☐ Todos os integrantes sabem responder sobre qualquer parte do código
- ☐ **Repositório, relatório e slides entregues no prazo**

Resumo: o que entregar e quando

Dia	Entregável principal	Onde
3	Projeto rodando + primeira pergunta ao agente	Git
8	Agente completo (todas as tools + 1 extra)	Git (tag v1.0-agente)
10	<code>benchmark.json</code> com 30+ perguntas	Git
12	Resultados do benchmark + análise inicial	Git (<code>logs/</code>)
14	Relatório em PDF + slides preenchidos	Git
15	Apresentação	Sala de aula

Quando pedir ajuda

Situação	Quando agir
Ambiente não roda / dependências quebrando	Dia 1 mesmo
API key não funciona / créditos esgotaram	Imediatamente
Travados na implementação de uma tool	Após 3h tentando
Agente não chama as tools certas	Dia 6–7 (durante stress test)
Benchmark com acurácia muito baixa ($< 40\%$)	Dia 11
Dúvida conceitual sobre o que mostrar no relatório	Dia 13

Canais: canal da disciplina (preferencial), e-mail do professor para questões urgentes.

Bom trabalho.